

Recursive Sorting

CSCI 1101

2024-04-19

Recursive List Sorting

There are many possible *algorithms* for sorting a list.

Today we will study two of them that can be described recursively.

Quicksort

The **quicksort** algorithm for sorting a list is described by the following specification:

- A list with fewer than two items is already sorted.
- To sort a list with at least two items, do the following:
 - take the *head* element of the list to be the **pivot**,
 - take the *tail* of the list and divide it into two lists as follows:
a **lower list** containing the elements less than or equal to the pivot element
and an **upper list** containing the elements greater than the pivot element.
 - *sort* each of the lower and upper lists,
 - the sorted list will be the result of concatenating three lists:
first, the sorted lower list
next, the list containing only the pivot element
finally, the sorted upper list.

Quicksort (activity)

Let's see how this works in practice.

Once we have a sense of how this algorithm works, let's try to write it in python:

```
def quick_sort(items) :  
    ''' sig: (list int) --> list int '''
```

Recursive List Merging

To write our next sorting function we need a function to **merge** two sorted lists into one sorted list.

Here is a specification for merging sorted lists xs and ys :

- If either list is empty then just concatenate them together.
- If neither list is empty then compare their head elements.
 - If the head of xs is less than the head of ys then the result is the list obtained by merging the tail of xs with all of ys , then inserting the head of xs at the beginning.
 - Otherwise, the result is the list obtained by merging all of xs with the tail of ys , then inserting the head of ys at the beginning.

Merging (activity)

Let's see how this works in practice.

Once we have a sense of how this algorithm works, let's try to write it in python:

```
def merge(xs , ys) :  
    ''' sig: (list int , list int) --> list int '''
```

Mergesort

The **mergesort** algorithm for sorting a list is described by the following specification:

- A list with fewer than two elements is already sorted.
- To sort a list with at least two items, do the following:
 - divide the list into two lists as follows:
the first list is the slice from the beginning to the midpoint and
the second list is the slice from the midpoint to the end.
 - sort each of these two lists,
 - *merge* the two sorted lists back together into one sorted list.

Mergesort (activity)

Let's see how this works in practice.

Once we have a sense of how this algorithm works, let's try to write it in python:

```
def merge_sort(items) :  
    ''' sig: (list int) --> list int '''
```

To Do

Finish *Project 3 - Automated Grader (Part 2)* on CodeRunner by midnight tonight.

Finish *Homework 11: Recursion* on CodeRunner by 11:00 next Thursday.